# Titre du Projet

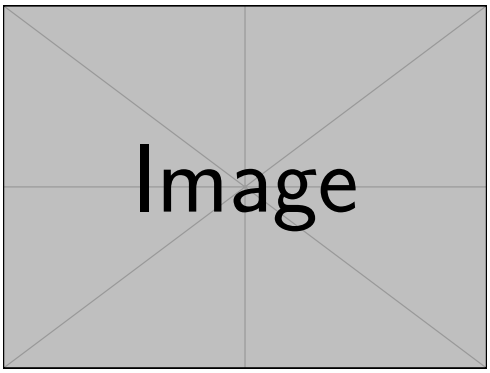
Nom de l'étudiant 1

Nom de l'étudiant 2

Année universitaire 2025–2026

Établissement :

Enseignante :




date

# Table des matières


**1 Introduction et présentation de l'idée** **3**

1.1 Contexte et objectifs . . . . . 3

1.2 L'idée originale . . . . . 3

1.3 Défense de l'idée (justification) . . . . . 3

**2 Conception** **4**

2.1 Diagramme des cas d'utilisation (UC) principal . . . . . 4

2.2 Détail des cas d'utilisation (UC détaillés) . . . . . 4

2.2.1 UC1 : Créer une tâche . . . . . 4

2.2.2 UC2 : Créer une tâche . . . . . 4

2.3 Diagrammes de séquence . . . . . 5

2.3.1 Titre de séq 1 . . . . . 5

2.3.2 Titre de séq 2 . . . . . 5

2.4 Diagrammes supplémentaires (classe, composant, entité-association) . . . . 5

2.4.1 Diagramme de classes (si le projet l'impose) . . . . . 5

2.5 Architecture technique (front-end/back-end/BD) . . . . . 5

2.6 Maquettes / wireframes . . . . . 5

**3 Développement** **7**

3.1 Technologies choisies et justifications . . . . . 7

3.2 Organisation du code . . . . . 7

3.3 Implémentation des fonctionnalités clés . . . . . 8

3.3.1 Création d'une tâche . . . . . 8

3.4 Gestion des données . . . . . 8

3.4.1 Modèle de données . . . . . 9

3.4.2 Requête importante . . . . . 9

**4 Tests et validation** **10**

4.1 Environnement et outils de test . . . . . 10

4.2 Scénarios de test . . . . . 10

**5 Conclusion et perspectives** **11**

5.1 Bilan du projet . . . . . 11

5.2 Difficultés globales et apprentissages . . . . . 11
5.3 Évolutions possibles . . . . . 11

**6 Annexes** **12**
6.1 Lien vers le dépôt Git . . . . . 12
6.2 Captures d'écran complémentaires . . . . . 12
6.3 Fichiers de configuration . . . . . 12

# 1 Introduction et présentation de l'idée

## 1.1 Contexte et objectifs

Présentation succincte du sujet, du cadre du projet. Expliquer le domaine d'application, les enjeux et ce qui motive le choix.

## 1.2 L'idée originale

Décrire l'idée : quel problème résout-elle ? Pour qui ? Quelle valeur ajoutée ?
*Exemple :* Notre projet consiste à développer une application web de gestion de tâches collaboratives avec un système de priorisation visuelle.

## 1.3 Défense de l'idée (justification)

Pourquoi ce projet plutôt qu'un autre ? Quels sont les avantages visés (simplicité, originalité, utilité, expérience utilisateur, etc.) ? Argumenter pour convaincre de la pertinence du choix.

# 2 Conception

## 2.1 Diagramme des cas d'utilisation (UC) principal

Vue d'ensemble des acteurs et des fonctionnalités principales. Commentaire sur les UC essentiels.

`Diagramme des cas d'utilisation avec commentaire`

## 2.2 Détail des cas d'utilisation (UC détaillés)

Pour 2 ou 3 UC les plus importants : description textuelle (préconditions, postconditions, scénario nominal, scénarios alternatifs).

### 2.2.1 UC1 : Créer une tâche

- **Préconditions** : Utilisateur connecté.
- **Postconditions** : Une nouvelle tâche est ajoutée au tableau.
- **Scénario nominal** : 1. L'utilisateur clique sur "Ajouter une tâche". 2. Il saisit le titre et la description. 3. Il valide. 4. La tâche apparaît dans la colonne "À faire".
- **Scénarios alternatifs** : Si les champs obligatoires sont vides, un message d'erreur s'affiche.

### 2.2.2 UC2 : Créer une tâche

- **Préconditions** : Utilisateur connecté.
- **Postconditions** : Une nouvelle tâche est ajoutée au tableau.
- **Scénario nominal** : 1. L'utilisateur clique sur "supprimer une tâche". 2. Il saisit le titre et la description. 3. Il valide. 4. La tâche est supprimée
- **Scénarios alternatifs** : Si les champs obligatoires sont vides, un message d'erreur s'affiche.

## 2.3 Diagrammes de séquence

Un diagramme par scénario clé (exemple : inscription, création de contenu, recherche, etc.). Permet d'illustrer les interactions entre les composants.

### 2.3.1 Titre de séq 1

— **Premier diagramme avec commentaire**

### 2.3.2 Titre de séq 2

— **Deuxième diagramme avec commentaire**

## 2.4 Diagrammes supplémentaires (classe, composant, entité-association)

Choisir les diagrammes pertinents pour le projet.

### 2.4.1 Diagramme de classes (si le projet l'impose)

`Diagramme de classes doit illustrer toutes les classes avec leurs cardinalités`

## 2.5 Architecture technique (front-end/back-end/BD)

Schéma d'architecture montrant les composants : client (navigateur), serveur web, serveur d'application, base de données. Indication des technologies prévues (langages) et des flux de données.

`schéma clair et compréhensible`

## 2.6 Maquettes / wireframes

2 à 3 écrans principaux avec commentaires et annotations sur les choix ergonomiques.

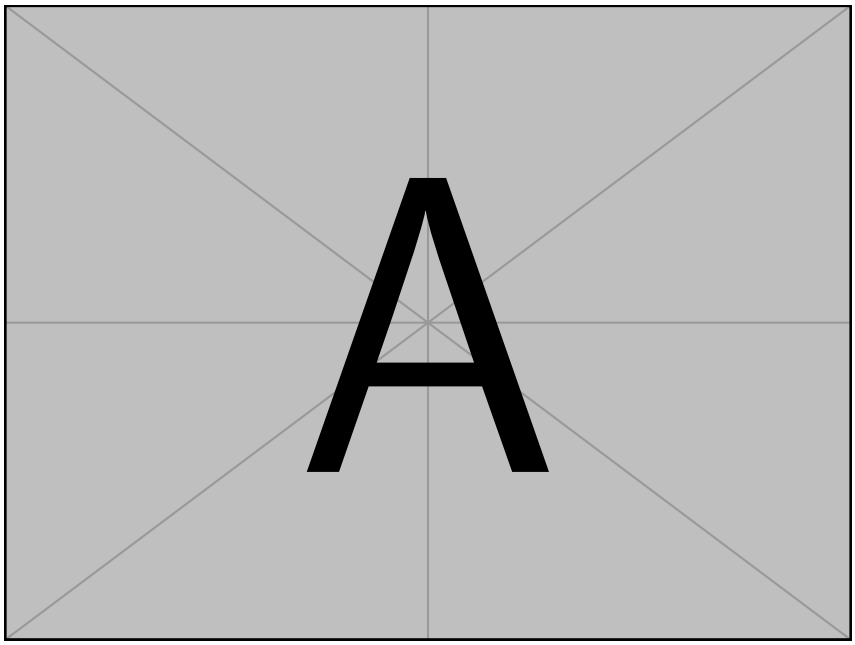



FIGURE 2.1 – Maquette de l'écran principal

# 3 Développement

## 3.1 Technologies choisies et justifications

Liste des langages et outils. Pour chaque choix, une justification technique en lien avec l'idée du projet.

- **Front-end : "Nom de la technologie"** – rôle de cette technologie dans le projet
- **Back-end : .....** – .....
- **Base de données :.....** – .....

## 3.2 Organisation du code

Arborescence commentée du projet (front, back, configuration, etc.). Explication de la structure (séparation des préoccupations, modularité). Voici un exemple :

```
projet/
          frontend/
                public/
                src/
                      components/
                      pages/
                      services/
                      App.js
                package.json
          backend/
                models/
                controllers/
                routes/
                config/
                server.js
          README.md
```

Listing 3.1 – Arborescence du projet

## 3.3 Implémentation des fonctionnalités clés

Pour 3–4 fonctionnalités importantes :

- Description de la logique
- Extrait de code pertinent (commenté)
- Difficulté rencontrée et solution adoptée
- Lien avec l'idée défendue

### 3.3.1 Création d'une tâche

*Logique :* L'utilisateur saisit les données, une requête POST est envoyée au back-end, qui enregistre en base. Voici un exemple :

```
// backend/controllers/taskController.js
exports.createTask = async (req, res) => {
    try {
        const { title, description } = req.body;
        // Validation simple
        if (!title) {
            return res.status(400).json({ message: 'Le titre est requis' });
        }
        const newTask = new Task({ title, description, user: req.userId });
        await newTask.save();
        res.status(201).json(newTask);
    } catch (error) {
        res.status(500).json({ message: 'Erreur serveur' });
    }
};
```

Listing 3.2 – Extrait du contrôleur de création de tâche

*Difficulté :* Gestion des erreurs asynchrones. *Solution :* Utilisation de try/catch et d'un middleware global d'erreur.

## 3.4 Gestion des données

Schéma relationnel avec explications. Exemples de requêtes importantes.

### 3.4.1 Modèle de données

### 3.4.2 Requête importante

Récupérer toutes les tâches d'un utilisateur triées par date :

```sql
SELECT * FROM tasks WHERE user_id = 1 ORDER BY created_at DESC;
```

Listing 3.3 – Requête SQL équivalente

# 4 Tests et validation

## 4.1 Environnement et outils de test

Navigateurs testés (Chrome, Firefox, Safari), validateurs HTML/CSS, éventuels tests unitaires.

## 4.2 Scénarios de test

Tableau récapitulatif des fonctionnalités avec statut (OK/KO/corrigé).

| **Fonctionnalité** | **Statut** | **Observations** |
|---|---|---|
| Inscription | OK | |
| Connexion | OK | |
| Création tâche | OK | |
| Modification tâche | OK | |
| Suppression tâche | KO | Corrigé après correction de l'ID |

Table 4.1 – Résultats des tests fonctionnels

# 5 Conclusion et perspectives

## 5.1 Bilan du projet

Ce qui a été réalisé par rapport à l'idée initiale. Le projet a atteint ses objectifs principaux : une application fonctionnelle de gestion de tâches.

## 5.2 Difficultés globales et apprentissages

Principaux obstacles techniques (gestion des sessions,.....) ou organisationnels (communication dans l'équipe). Compétences acquises en cette matière, travail collaboratif.

## 5.3 Évolutions possibles

Fonctionnalités supplémentaires (notifications, partage de tableaux), optimisation (mise en cache), déploiement sur un serveur cloud.

# 6 Annexes

## 6.1 Lien vers le dépôt Git

https://github.com/etudiant/projet-exemple (obligatoire)

## 6.2 Captures d'écran complémentaires

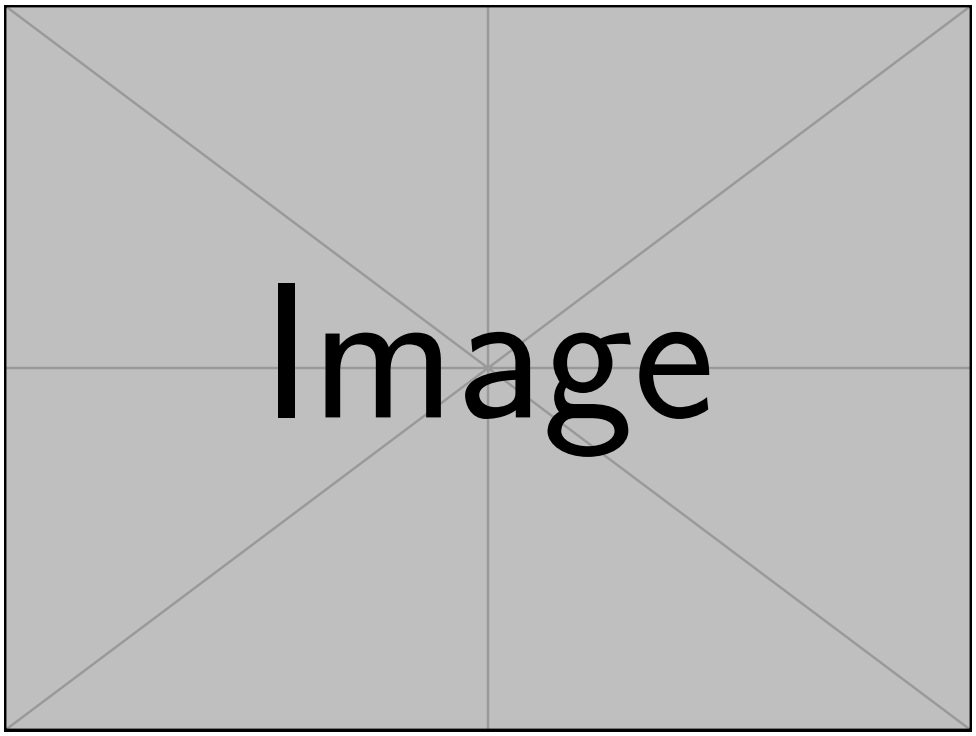



FIGURE 6.1 – Capture de l'application

## 6.3 Fichiers de configuration

Exemples de fichiers de configuration (package.json, .env, etc.)

```
{
  "name": "projet-tasks",
  "version": "1.0.0",
```

```
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.0"
  }
}
```

Listing 6.1 – package.json (extrait)